ROBOT MOVEMENT SIMULATOR

C++ Based Robot Navigation and BFS Path Planning

Project Documentation

Name: Jayasri.D
College Name : Annai Mathammal Sheela Engineering College
Academic Year: 2023–2027
Branch: B.E. Robotics and Automation
Programming Language: C++
Development Environment: Visual Studio Code
Project Type: Robotics Simulation

No.	Topic	Page No.
	ABSTRACT	3
1	INTRODUCTION	4
2	PROBLEM STATEMENT	5
3	OBJECTIVES	6
4	TECHNOLOGIES USED	7
5	SYSTEM DESIGN	8
6	ROBOT MOVEMENT AND DIRECTION CONTROL	9
7	SENSOR SYSTEM	11
8	BFS PATH PLANNING	14
9	DISTANCE CALCULATION	16
10	AUTOMATIC NAVIGATION	18
11	USER COMMANDS	21
12	COMMAND HISTORY AND STATISTICS	25
13	FINAL REPORT / SIMULATION OUTPUT	28
14	TESTING AND RESULTS	31
15	SCREENSHOTS	35
16	CONCLUSION	39

ABSTRACT

The Robot Movement Simulator is a C++-based robotics simulation project designed to demonstrate basic robot navigation and path planning in a grid environment. The robot operates within a 5×5 grid containing predefined obstacles and boundaries. It can move forward and backward, change its direction, scan its surroundings using simulated sensors, and navigate automatically toward a predefined goal.

The project uses the Breadth-First Search (BFS) algorithm to determine the shortest valid movement path while avoiding obstacles. The simulator also records movement history, collision statistics, robot position, direction, and final navigation results. Automatic navigation allows the robot to follow the calculated path until it reaches the goal.

This project demonstrates fundamental concepts in robotics, C++ programming, grid-based navigation, sensor logic, and path-planning algorithms.

1. INTRODUCTION

Robotics is an important field that combines programming, electronics, sensors, and mechanical systems to develop intelligent machines. One of the basic tasks of a robot is to move from one location to another while avoiding obstacles and staying within a defined environment.

This project, Robot Movement Simulator, is developed using C++ to understand the basic concepts of robot movement and navigation without using a physical robot. The robot is represented inside a 5×5 grid environment with fixed obstacles and boundaries. It starts from the position (0,0) and has to reach the goal at (4,4).

The simulator provides different commands such as forward, backward, left, and right for controlling the robot. It also includes a simulated sensor system that checks the surroundings of the robot for obstacles and boundaries based on its current direction.

For path planning, the project uses the Breadth-First Search (BFS) algorithm. BFS helps the robot find a shortest valid path to the goal while avoiding obstacles. The project also includes an automatic navigation mode where the robot follows the calculated path step by step.

Overall, this project helped me understand how C++ programming can be applied to a robotics navigation problem. It also provided practical understanding of grid-based movement, sensor logic, path planning, and robot navigation.

2. PROBLEM STATEMENT

In a grid-based environment, a robot needs to move from a starting position to a specific destination while avoiding obstacles and boundaries. When controlling a robot, simply giving movement commands is not enough because the robot must also consider its current direction and the conditions around it.

The problem addressed in this project is to create a simulation where a robot can move within a fixed 5×5 grid, detect obstacles and boundaries using simulated sensors, and reach a predefined goal. The simulator should also be able to determine a shortest valid path and provide automatic navigation based on that path.

The project uses the Breadth-First Search (BFS) algorithm to solve the path-planning problem while considering the obstacles present in the environment. The simulator also records movement information and navigation results to make the robot's behaviour easier to observe and understand.

3. OBJECTIVES

- To develop a C++-based robot navigation simulator for demonstrating fundamental robotic movement and navigation concepts.
- To model robot movement within a 5×5 grid environment containing predefined obstacles and boundaries.
- To implement directional movement and orientation control using forward, backward, left, and right commands.
- To design a simulated sensor system for identifying obstacles and boundary conditions around the robot.
- To apply the Breadth-First Search (BFS) algorithm to determine the shortest valid path from the robot's current position to the predefined goal.
- To implement an automatic navigation mechanism that enables the robot to follow the calculated path.
- To maintain and display relevant navigation information, including movement count, collision statistics, command history, and robot path.
- To provide a distance-to-goal calculation based on the shortest valid path available in the environment.
- To improve understanding of C++ programming, data structures, algorithms, and their application in robotics navigation.
- To provide a practical foundation for further learning in robot path planning and autonomous navigation.

4. TECHNOLOGIES USED

4.1 Programming Language

C++ was used as the primary programming language for developing the Robot Movement Simulator. C++ provides the required programming features for implementing robot movement, sensor logic, path planning, and data handling
.

4.2 Development Environment

Visual Studio Code was used as the development environment for writing, compiling, debugging, and executing the C++ program.

4.3 Algorithm

The Breadth-First Search (BFS) algorithm was used for path planning. It helps determine the shortest valid path between the robot's current position and the goal while considering obstacles and grid boundaries.

4.4 Data Structures

The project uses C++ data structures such as vectors, queues, pairs, and structures to store grid information, obstacle positions, robot states, paths, and navigation data.

4.5 Simulation Environment

A 5×5 grid-based environment is used to represent the robot's operating area. The environment contains the robot, predefined goal, obstacles, and boundaries.

5. SYSTEM DESIGN

The Robot Movement Simulator is designed as a grid-based navigation system. The complete system consists of several components that work together to control the robot, monitor its surroundings, calculate paths, and provide navigation results.

5.1 Grid Environment

The robot operates within a 5×5 grid, where the X and Y coordinates range from 0 to 4. The robot starts at (0,0) and the predefined goal is at (4,4).

Three fixed obstacles are placed at (2,2), (4,1), and (1,3). The robot is not allowed to move through these positions or outside the grid boundaries.

5.2 Robot Position and Direction

The simulator maintains the robot's current position and orientation throughout the navigation process. The robot can face four directions: North (N), East (E), South (S), and West (W).

The left and right commands change only the robot's orientation. They do not change its position. The current direction is used to determine how the robot moves forward or backward and how its sensors scan the surrounding positions.

5.3 Movement Control

The robot can be controlled using the forward, backward, left, and right commands. When a movement command is given, the simulator calculates the target position based on the robot's current direction.

Before allowing the movement, the system checks whether the target position is inside the grid and whether it contains an obstacle. If the position is valid, the robot moves to that position. Otherwise, the movement is prevented and the corresponding obstacle or boundary collision is recorded.

5.4 Path Planning

The system uses the Breadth-First Search (BFS) algorithm for path planning. BFS checks the possible valid positions from the robot's current location and determines a shortest valid path to the goal.

The calculated path can be viewed using the path command. The same path-planning mechanism is also used by the automatic navigation feature

5.5 Navigation Monitoring

During the simulation, the system continuously maintains important information such as the robot's position, direction, distance to the goal, movement count, sensor status, and collision information.

This information allows the user to observe how the robot behaves within the environment and understand the result of each navigation command.

6. ROBOT MOVEMENT AND DIRECTION CONTROL

The robot movement system is one of the main components of the simulator. It allows the robot to move within the 5×5 grid while maintaining its current position and direction. The movement system also checks obstacles and grid boundaries before allowing the robot to move.

6.1 5×5 Grid Environment

The simulator uses a 5×5 grid with coordinates ranging from 0 to 4 on both the X-axis and Y-axis. The robot starts at (0,0) and the predefined goal is located at (4,4).

The grid provides a fixed environment in which the robot can perform its navigation. Each movement changes the robot's position by one grid cell.

6.2 Forward Movement

The forward command moves the robot one position in the direction it is currently facing.

For example, if the robot is at (0,0) and facing North, one forward movement changes its position to (0,1). If the robot is facing East, the same command changes its position by one step along the X-axis.

Before every movement, the simulator checks the target position for obstacles and boundaries.

6.3 Backward Movement

The backward command moves the robot one grid position opposite to its current direction.

For example, when the robot is facing North, the backward command attempts to move it toward the South. The simulator performs the same obstacle and boundary checks before completing the movement.

6.4 Left and Right Rotation

The left and right commands change only the robot's orientation. The robot does not change its position when turning.

The direction changes according to the following sequence:

Right:
N → E → S → W → N

Left:
N → W → S → E → N

The current direction is important because it determines the movement direction for the next forward or backward command and also controls the orientation of the sensor scan.

6.5 Obstacle Detection

The simulator contains three predefined obstacles at:

- (2,2)
- (4,1)
- (1,3)

When the robot attempts to move into an obstacle position, the movement is prevented. The simulator displays an obstacle detection message and records the event as an obstacle collision.

6.6 Boundary Detection

The robot is not allowed to move outside the 5×5 grid. If a movement command attempts to take the robot outside the valid coordinate range, the movement is prevented.

The simulator displays “Boundary reached!” and records the event as a boundary collision. The robot remains at its current position.

6.7 Movement Tracking

Every successful movement is recorded in the robot's path. The simulator also maintains a movement count to track the number of successful movements performed during the simulation.

Obstacle and boundary collisions are recorded separately, allowing the final navigation performance to be analysed.

7. SENSOR SYSTEM

The Robot Movement Simulator contains a simulated sensor system to check the positions surrounding the robot. The sensor system works based on the robot's current position and direction. It does not use any physical sensor hardware. Instead, the program calculates the coordinates around the robot and checks the condition of each position.

7.1 Four-Direction Sensor Scan

The simulator checks four relative positions around the robot::

- Front
- Back
- Left
- Right

These positions are not permanently fixed to the screen. They are calculated according to the direction in which the robot is currently facing.

For example, when the robot is facing North, the position in front of the robot is the position with an increased Y-coordinate. When the robot turns to East, the front position changes to the position with an increased X-coordinate.

Therefore, the sensor system always considers the robot's current orientation while performing the scan.

7.2 Front Sensor

The Front sensor checks the grid position directly in front of the robot according to its current direction.

If the position is within the grid and does not contain an obstacle, the sensor reports:

Front: CLEAR

If an obstacle is present at that position, it reports the obstacle condition along with its coordinate.

If the position is outside the grid, the sensor reports:

Front: BOUNDARY

7.3 Back Sensor

The Back sensor checks the position directly behind the robot. Its position is calculated as the opposite direction of the robot's current orientation.

For example, when the robot is facing North, the back position is toward South. If that position contains an obstacle, the sensor identifies it. If it falls outside the grid, it is reported as a boundary.

7.4 Left and Right Sensors

The Left and Right sensors check the positions to the respective sides of the robot based on its current direction.

For example, when the robot is facing North:

- Front → North
- Back → South
- Left → West
- Right → East

If the robot turns to East, these relative positions are recalculated according to the new orientation.

7.5 Sensor Condition Checking

After calculating the four sensor positions, the simulator checks each position using the grid information.

The position can have one of three conditions:

1. CLEAR
The position is inside the grid and does not contain an obstacle.

2. OBSTACLE
The position contains one of the predefined obstacles.

3. BOUNDARY
The calculated position lies outside the valid 0–4 grid range.

This checking is performed separately for Front, Back, Left, and Right.

7.6 Sensor Scan Output

The sensor scan is displayed after the robot performs a movement or rotation. The output shows the condition detected in all four relative directions.

For an obstacle, the coordinate is also displayed, such as:

OBSTACLE at (2,2)

This makes it possible to observe what the robot's simulated sensors detect around its current position.

7.7 Relationship Between Direction and Sensors

The robot's direction is important to the sensor system because the sensor positions are calculated relative to the robot.

For example:

Robot facing N → Front = North

After a right turn:

Robot facing E → Front = East

The robot's position does not change when turning, but the sensor directions change because its orientation has changed.

Thus, the sensor system provides a direction-based representation of the robot's immediate surroundings within the 5×5 grid.

8. BFS PATH PLANNING

The Robot Movement Simulator uses the Breadth-First Search (BFS) algorithm to find a shortest valid path between the robot's current position and the predefined goal. The BFS calculation considers the available grid positions, obstacles, and grid boundaries.

8.1 Purpose of BFS

The main purpose of BFS in this project is to determine the shortest valid movement path from the robot's current position to the goal.

The BFS path calculation considers the position of the robot, while the robot's direction is handled separately during navigation.

8.2 Starting the Search

When BFS is called, the robot's current (x,y) position is taken as the starting position and (4,4) is used as the goal.

The starting position is added to a queue. BFS then begins checking the valid neighbouring grid positions.

8.3 Checking Neighbouring Positions

For every position processed by BFS, the algorithm checks the possible neighbouring positions.

A position can be considered for the path only when:

- It is inside the 5×5 grid.
- It is not one of the predefined obstacles.
- It has not already been visited.

Valid positions are added to the queue for further exploration.

8.4 Visited Positions

The algorithm keeps track of visited positions so that the same grid position is not processed repeatedly.

The BFS implementation also stores the previous position from which a new position was reached. This information is required to reconstruct the path after the goal is found.

8.5 Finding the Shortest Path

BFS explores the grid level by level. Since each valid movement between neighbouring grid positions has the same cost, the path found by BFS provides the shortest valid movement path for the current environment.

The search continues until the goal (4,4) is reached.

8.6 Path Reconstruction

After reaching the goal, the stored previous-position information is used to trace the path back from the goal to the robot's starting position.

The path is then arranged in the correct order:

- Starting Position → Intermediate Positions → Goal

This calculated path is displayed when the user uses the path command.

8.7 Distance Calculation Using BFS

The project also uses the BFS result to calculate the Distance to Goal.

The distance represents the number of valid grid movements required to reach the goal from the robot's current position while considering the obstacles and boundaries.

For example, when the robot is at (0,0), the calculated distance is based on the shortest valid BFS path rather than simply calculating the direct Manhattan distance.

8.8 BFS in Automatic Navigation

The auto command uses BFS-based path planning to navigate the robot toward the goal.

From the robot's current position, the simulator calculates a valid path to the goal. It then considers the next position in that path, changes the robot's direction when required, and moves the robot forward.

This process continues until the robot reaches (4,4).

9. DISTANCE CALCULATION

The Robot Movement Simulator calculates the Distance to Goal using the Breadth-First Search (BFS) algorithm. The purpose of this calculation is to determine the minimum number of valid movements required for the robot to reach the predefined goal from its current position.

9.1 Purpose of Distance Calculation

The distance calculation provides information about the number of valid grid movements required for the robot to reach the goal. It helps the user understand the robot's current navigation state based on the available path in the grid environment.

The distance is not calculated using only the difference between the X and Y coordinates. Instead, the simulator uses BFS to consider the actual grid environment, including predefined obstacles and grid boundaries.

9.2 BFS-Based Distance Calculation

When the distance is calculated, the simulator performs a BFS search starting from the robot's current position. The goal position is fixed at (4,4).

BFS explores valid neighbouring positions while avoiding the predefined obstacles and positions outside the 5×5 grid. The shortest valid path obtained from this search is used to determine the distance to the goal.

9.3 Number of Movements

The calculated distance represents the number of valid movements required to travel from the robot's current position to the goal.

For example, if the shortest valid BFS path requires 8 movements, the simulator displays:

Distance to Goal: 8

Each movement from one valid grid position to an adjacent valid position is counted as one step.

9.4 Effect of Obstacles and Boundaries

The distance calculation considers the obstacles and boundaries present in the simulation. Therefore, the shortest valid route may differ from a direct route based only on the coordinates.

If an obstacle blocks a possible route, BFS searches through other valid grid positions. Positions outside the 5×5 grid are also excluded from the path calculation.

9.5 Distance Based on Current Position

The distance is calculated from the robot's current position. Therefore, when the robot moves to another position, the shortest valid distance to the goal is recalculated based on the new position.

The BFS distance calculation considers the robot's position, while its direction is handled separately by the movement and automatic navigation system.

9.6 Distance at the Goal

When the robot reaches the goal position (4,4), no further movement is required. Therefore, the distance to the goal becomes 0.

The robot status displays:

Distance to Goal: 0

9.7 Unreachable Goal

If BFS cannot find a valid path from the robot's current position to the goal, the distance calculation returns -1. This indicates that the goal cannot be reached from the current position through the available valid grid positions.

9.8 Distance Display in Robot Status

The calculated distance is displayed as part of the robot status information along with the robot's current position, direction, goal position, and movement count.

For example:

Robot Position: (3,2)
Direction: E
Goal: (4,4)
Distance to Goal: 3

From (3,2), the shortest valid route is (3,2) → (4,2) → (4,3) → (4,4), which requires three movements.

This provides the user with a clear indication of the robot's current position and the minimum valid movement distance to the goal.

10. AUTOMATIC NAVIGATION

The Robot Movement Simulator includes an automatic navigation system that enables the robot to move from its current position toward the predefined goal without requiring the user to manually enter every movement command. The automatic navigation process is started using the auto command.

The system combines BFS-based path planning, direction control, forward movement, and simulated sensor scanning to guide the robot through the grid environment.

10.1 Starting Automatic Navigation

When the user enters the auto command, the simulator starts the automatic navigation process from the robot's current position.

The robot does not simply move directly toward the goal. Instead, the simulator first calculates a valid path using the Breadth-First Search (BFS) algorithm. The path calculation considers the current robot position, predefined obstacles, and the boundaries of the 5×5 grid.

10.2 Calculating the Navigation Path

BFS searches the grid from the robot's current position until it reaches the goal (4,4). During this search, positions containing obstacles and positions outside the grid are excluded.

Since each movement between neighbouring grid positions has the same cost, BFS provides the shortest valid movement path for the current environment.

The resulting path contains the sequence of grid positions that the robot needs to follow to reach the goal.

10.3 Identifying the Next Position

After obtaining the BFS path, the simulator identifies the next position that the robot needs to reach.

The robot compares its current position with the next position in the path. Based on this difference, the required movement direction is determined.

For example, if the robot is at (0,0) and the next position is (1,0), the robot needs to face East (E) before moving forward.

10.4 Direction Control During Automatic Navigation

The robot's direction is controlled separately from its position. If the robot is not facing the required direction, the simulator changes its direction before performing the movement.

The robot can face four directions:

- N → North
- E → East
- S → South
- W → West

Changing direction does not move the robot to another grid position. The position changes only when the forward movement is performed.

10.5 Sensor Scan Before Movement

Before each automatic movement, the simulator performs a sensor scan using the robot's current position and direction.
The sensor checks four relative positions:

- Front
- Back
- Left
- Right

Each position is reported as CLEAR, OBSTACLE, or BOUNDARY depending on the surrounding grid condition.

The automatic navigation system uses BFS to determine the next position and adjusts the robot's direction accordingly before performing the forward movement. The sensor scan provides information about the robot's immediate surroundings but does not determine the BFS path.

10.6 Moving to the Next Position

After the required direction has been set and the sensor scan has been performed, the robot moves forward to the next position in the BFS path.

The successful movement is added to the robot's path and the movement count is increased. The simulator then updates the robot's position and displays the updated navigation information.

10.7 Recalculating BFS

The automatic navigation process calculates BFS from the robot's current position during the navigation process. After the robot moves to a new position, the simulator determines the next part of the route based on the updated position.

This allows the robot to continue following a valid shortest path toward the goal.

10.8 Navigation Until Goal

The process of calculating the path, identifying the next position, adjusting the direction, scanning the surroundings, and moving forward continues until the robot reaches the predefined goal position (4,4).
When the robot reaches the goal, the simulator displays:

*** GOAL REACHED! ***

The automatic navigation process then ends.

10.9 Automatic Navigation Example

In the tested simulation, the robot started at (0,0) and automatically navigated to the goal at (4,4).

The successful navigation path was:

(0,0) → (1,0) → (2,0) → (3,0) → (3,1) → (3,2) → (4,2) → (4,3) → (4,4)

The robot completed this route using 8 successful movements. During the tested automatic navigation, there were 0 obstacle collisions and 0 boundary collisions.

10.10 Overall Automatic Navigation Flow

The automatic navigation process can be summarized as:

Current Position → BFS Path Calculation → Identify Next Position → Adjust Direction → Sensor Scan → Move Forward → Update Position → Continue Navigation → Reach Goal

This process demonstrates how path planning, robot orientation, movement control, and simulated environmental sensing can work together in a grid-based robotics navigation system.

11. USER COMMANDS

The Robot Movement Simulator provides a command-based interface through which the user can control the robot, obtain navigation information, perform automatic navigation, and manage the simulation. Commands are entered through the console and processed according to their respective functions.

11.1 Forward Command

The forward command moves the robot one grid position in its current direction.

Before moving, the simulator checks the target position for grid boundaries and predefined obstacles. If the position is valid, the robot moves to that position, the movement is added to the path, and the movement count is increased.

If an obstacle blocks the movement, the robot remains in its current position and displays:

Obstacle detected at (x,y)!

If the target position is outside the grid, the robot remains in its current position and displays:

Boundary reached!

11.2 Backward Command

The backward command moves the robot one grid position opposite to its current direction.

The simulator performs obstacle and boundary checks before allowing the movement. A successful movement is added to the robot's path and movement count. If movement is prevented by an obstacle or boundary, the corresponding collision is recorded.

11.3 Left Command

The left command rotates the robot without changing its current position.

The direction changes according to the following sequence:

N → W → S → E → N

The updated direction is used for subsequent movement and sensor scanning.

11.4 Right Command

The right command rotates the robot without changing its current position.

The direction changes according to the following sequence:

N → E → S → W → N

The updated direction is used for subsequent movement and sensor scanning.

11.5 Reset Command

The reset command returns the robot to its initial state. The robot is placed at (0,0) and faces North (N).

The robot's path is reset with (0,0) as the starting position. The movement count, obstacle collision count, and boundary collision count are reset. The command history is also cleared.

The grid is then displayed again from the initial state.

11.6 Help Command

The help command displays the available commands and their functions. It helps the user understand the operations supported by the simulator.

The available commands are:

- forward – Move forward
- backward – Move backward
- left – Turn left
- right – Turn right
- reset – Reset the robot
- help – Show commands
- path – Show BFS shortest path
- auto – Start automatic navigation
- stop – Stop the simulation

11.7 Path Command

The path command uses BFS to calculate and display the shortest valid path from the robot's current position to the goal.

The displayed path contains the robot's current position, intermediate valid positions, and the goal position. The number of movements required for the calculated path is also displayed.

11.8 Auto Command

The auto command starts automatic navigation from the robot's current position.

The simulator repeatedly calculates a BFS path, identifies the next position, changes the robot's direction when required, and moves the robot forward. A sensor scan is performed before each automatic movement.

This process continues until the robot reaches the goal at (4,4).

11.9 Stop Command

The stop command stops the simulation and produces the final navigation result.

If the robot has reached the goal, the simulator displays:

*** GOAL REACHED! ***

Otherwise, it displays:

*** GOAL NOT REACHED! ***

When the goal has not been reached, the simulator also provides the suggested minimum-command route. The final report then displays the robot's final position, direction, goal status, collision statistics, movement count, command history, and final path.

11.10 Invalid Commands

If the user enters a command that is not supported by the simulator, the command is treated as an invalid command. No robot movement or rotation is performed, and the simulator continues to wait for another command.

11.11 Command History

The simulator maintains a history of relevant simulation commands. Commands such as forward, backward, left, right, auto, and stop can appear in the command history.

The help and path commands are information-related commands and are not added to the command history. When reset is executed, the existing command history is cleared.

12. COMMAND HISTORY AND STATISTICS

The Robot Movement Simulator maintains information about the robot's actions and navigation performance throughout the simulation. The system records relevant commands, successful movements, collision events, and the robot's movement path. This information is used to monitor the simulation and generate the final navigation report.

12.1 Command History

The simulator maintains a history of relevant simulation commands entered during the current run.

The commands that can be recorded include:

- forward
- backward
- left
- right
- auto
- stop

The help and path commands are not recorded in the command history because they are used to display information.

When the reset command is executed, the existing command history is cleared and a new simulation state is started.

12.2 Movement Count

The Movement Count represents the number of successful movements performed by the robot.

A movement is counted only when the robot successfully changes its position to another valid grid cell. A movement blocked by an obstacle or boundary does not increase the successful movement count.

The left and right commands also do not increase the movement count because they only change the robot's direction without changing its position.

12.3 Obstacle Collisions

The simulator maintains a separate count for Obstacle Collisions. When the robot attempts to move into one of the predefined obstacle positions, the movement is prevented and the obstacle collision count is increased.

The robot remains at its current position and the simulator displays:

Obstacle detected at (x,y)!

12.4 Boundary Collisions

The simulator also maintains a separate count for Boundary Collisions. When the robot attempts to move outside the valid 5×5 grid, the movement is prevented and the boundary collision count is increased.

The robot remains at its current position and the simulator displays:

Boundary reached!

12.5 Robot Path Tracking

The simulator maintains the robot's movement path during the simulation. The path starts with the robot's initial position (0,0).

Whenever the robot successfully moves to another valid position, the new position is added to the path. Therefore, the path represents the sequence of positions reached by the robot through successful movements.

Blocked obstacle and boundary attempts do not change the robot's actual position. These events are represented separately in the displayed path output.

12.6 Path Event Markers

The path output uses markers to identify unsuccessful movement attempts.

- [X] represents a movement attempt blocked by an obstacle.
- [B] represents a movement attempt blocked by a boundary.

These markers help distinguish blocked movement attempts from the robot's actual successful movement positions.

12.7 Final Statistics and Report

When the simulation is stopped, the simulator generates a final report containing important navigation information. The report includes:

- Final robot position
- Final direction
- Goal position and goal status
- Obstacle collision count
- Boundary collision count
- Movement count
- Command history
- Final path

This information provides a summary of the robot's navigation during the simulation.

12.8 Resetting History and Statistics

When the reset command is executed, the robot is returned to its initial position (0,0) and its direction is reset to North (N).

The movement count, obstacle collision count, boundary collision count, command history, and previous path are cleared. A new path is started from (0,0).

This allows the user to perform a new simulation independently of the previous run.

12.9 Tested Simulation Statistics

During the tested automatic navigation, the robot successfully travelled from (0,0) to (4,4).

- The recorded results were:
- Movement Count: 8
- Obstacle Collisions: 0
- Boundary Collisions: 0
- Command History: auto

The final successful path was

(0,0) → (1,0) → (2,0) → (3,0) → (3,1) → (3,2) → (4,2) → (4,3) → (4,4)
These results demonstrate that the robot successfully reached the goal without encountering an obstacle or boundary collision during the tested automatic navigation.

13. FINAL REPORT / SIMULATION OUTPUT

The Robot Movement Simulator generates a final report when the user stops the simulation. The report provides a summary of the robot's final state and the recorded navigation information from the simulation.

The final report includes the robot's position, direction, goal status, distance to the goal, movement statistics, command history, and final movement path.

13.1 Purpose of the Final Report

The main purpose of the final report is to provide a clear summary of the robot's navigation result. It allows the user to determine whether the robot reached the goal, how many successful movements were performed, whether any obstacle or boundary collisions occurred, which commands were used, and which path was followed.

13.2 Final Robot Position

The final report displays the robot's position when the simulation is stopped. The position is represented using the X and Y coordinates of the grid.

When the robot successfully reaches the goal, the final position is (4,4).

Example:

Final Position: (4,4)

13.3 Final Direction

The final report displays the robot's direction at the end of the simulation. The robot can face North (N), East (E), South (S), or West (W).

Direction is maintained separately from the robot's position. Therefore, the final direction represents the orientation of the robot when the simulation is stopped.

In the tested automatic navigation, the final direction was:

Final Direction: N

13.4 Goal and Goal Status

The predefined goal of the simulation is (4,4). The final report indicates whether the robot has successfully reached this position.

When the robot reaches the goal, the simulator displays:

*** GOAL REACHED! ***

If the simulation is stopped before reaching the goal, the simulator displays:

*** GOAL NOT REACHED! ***

This provides a clear indication of whether the navigation task was completed successfully.

13.5 Distance to Goal

The final report includes the Distance to Goal, which is calculated using the BFS-based shortest valid path.

When the robot reaches (4,4), the distance becomes 0, because no further movement is required.

In the tested successful navigation:

Distance to Goal: 0

13.6 Movement Statistics

The final report displays the statistics collected during the simulation.

Movement Count represents the number of successful movements performed by the robot.

Obstacle Collisions represents the number of movement attempts blocked by predefined obstacles.

Boundary Collisions represents the number of movement attempts blocked because the target position was outside the valid grid.

These statistics provide information about the robot's movement performance during navigation.

13.7 Command History

The final report displays the relevant commands recorded during the current simulation. This provides a record of the main commands used to control or navigate the robot.

For example, when automatic navigation is used, the command history can contain:

Auto

Commands such as help and path are not added to the command history because they are information-related commands.

13.8 Final Robot Path

The final report displays the robot's movement path during the simulation. The path starts from the robot's initial position and records the positions reached through successful movements.

If blocked movement attempts occurred, the path output can also represent them using:

- [X] – Obstacle-blocked movement
- [B] – Boundary-blocked movement

These markers help distinguish blocked movement attempts from successful movement positions.

13.9 Tested Automatic Navigation Result

The final report was verified using the automatic navigation mode. The robot started from (0,0) and successfully reached the predefined goal at (4,4).

The tested results were:

- Final Position: (4,4)
- Final Direction: N
- Goal: (4,4)
- Distance to Goal: 0

- Movement Count: 8
- Obstacle Collisions: 0
- Boundary Collisions: 0
- Goal Status: REACHED
- Command History: auto

13.10 Tested Final Path

The successful path obtained during automatic navigation was:

(0,0) → (1,0) → (2,0) → (3,0) → (3,1) → (3,2) → (4,2) → (4,3) → (4,4)

The robot completed the route using 8 successful movements. No obstacle collision or boundary collision occurred during this test.

13.11 Program Completion

After displaying the final navigation information, the simulator completes the current simulation and displays:

Program stopped.

This marks the end of the current simulation run.

13.12 Importance of the Final Report

The final report makes the simulation results easier to review and document. It combines the robot's final position, direction, goal status, distance, movement statistics, command history, and path into a single navigation summary.

This provides a clear record of the robot's performance within the simulated 5×5 grid environment.

14. TESTING AND RESULTS

Testing was performed to verify the different functions of the Robot Movement Simulator and to ensure that the robot behaves correctly within the 5×5 grid environment. The testing focused on movement, direction control, obstacle and boundary detection, BFS path planning, automatic navigation, reset, and final report generation.

14.1 Movement Testing

The forward and backward commands were tested to verify that the robot moves according to its current direction.

The robot was also tested after changing its direction using the left and right commands. The results confirmed that rotation changes the robot's direction without changing its position.

14.2 Obstacle Detection Testing

Movement toward predefined obstacle positions was tested to verify that the robot does not enter an obstacle cell.

When the robot attempts to move into an obstacle, the simulator displays:

Obstacle detected at (x,y)!

The robot remains in its current position and the obstacle collision count is increased.

14.3 Boundary Detection Testing

Movement beyond the valid 0–4 coordinate range was tested to verify boundary handling.

When the robot attempts to move outside the grid, the simulator displays:

Boundary reached!

The robot remains in its current position and the boundary collision count is increased.

14.4 Sensor Testing

The simulated sensor system was tested in different robot positions and directions. The sensor scan checks the Front, Back, Left, and Right positions relative to the robot's current orientation.

The sensor correctly reports the surrounding condition as:

- CLEAR

- OBSTACLE
- BOUNDARY

Direction changes were also tested to verify that the sensor positions are recalculated according to the new robot orientation.

14.5 BFS Path Testing

The BFS path-planning function was tested to verify that it calculates a shortest valid path while considering the predefined obstacles and grid boundaries.

The path command was used to display the calculated route from the robot's current position to the goal.

14.6 Distance Calculation Testing

The Distance to Goal calculation was tested from different robot positions. The distance was calculated using the BFS shortest valid path rather than only using the direct coordinate difference.

When the robot reaches the goal position (4,4), the distance is correctly displayed as:

Distance to Goal: 0

14.7 Automatic Navigation Testing

The auto command was tested to verify that the robot can automatically navigate from its current position to the goal using BFS-based path planning.

During automatic navigation, the robot determines the next position, adjusts its direction when required, performs a sensor scan, and moves forward.

14.8 Reset Testing

The reset command was tested to verify that the simulator returns to its initial state.

After reset:

Position: (0,0)
Direction: N

The movement count, obstacle collision count, boundary collision count, command history, and previous path are also reset.

14.9 Stop and Final Report Testing

The stop command was tested to verify that the simulator correctly identifies whether the goal has been reached and generates the final navigation report.

The final report was verified to contain the final position, direction, goal status, distance to goal, movement count, collision statistics, command history, and final path.

14.10 Automatic Navigation Test Result

The automatic navigation test successfully completed from the starting position (0,0) to the goal (4,4).

The verified result was:

- Final Position: (4,4)
- Final Direction: N
- Movement Count: 8
- Obstacle Collisions: 0
- Boundary Collisions: 0
- Distance to Goal: 0
- Goal Status: REACHED

14.11 Verified Path

The final path obtained during the automatic navigation test was:

(0,0) → (1,0) → (2,0) → (3,0) → (3,1) → (3,2) → (4,2) → (4,3) → (4,4)

The robot successfully reached the goal in 8 movements without any obstacle or boundary collisions.

14.12 Testing Summary

The testing confirmed that the major functions of the Robot Movement Simulator operate as intended. Robot movement, direction control, obstacle detection, boundary handling, sensor scanning, BFS path planning, distance calculation, automatic navigation, reset, and final report generation were successfully verified.

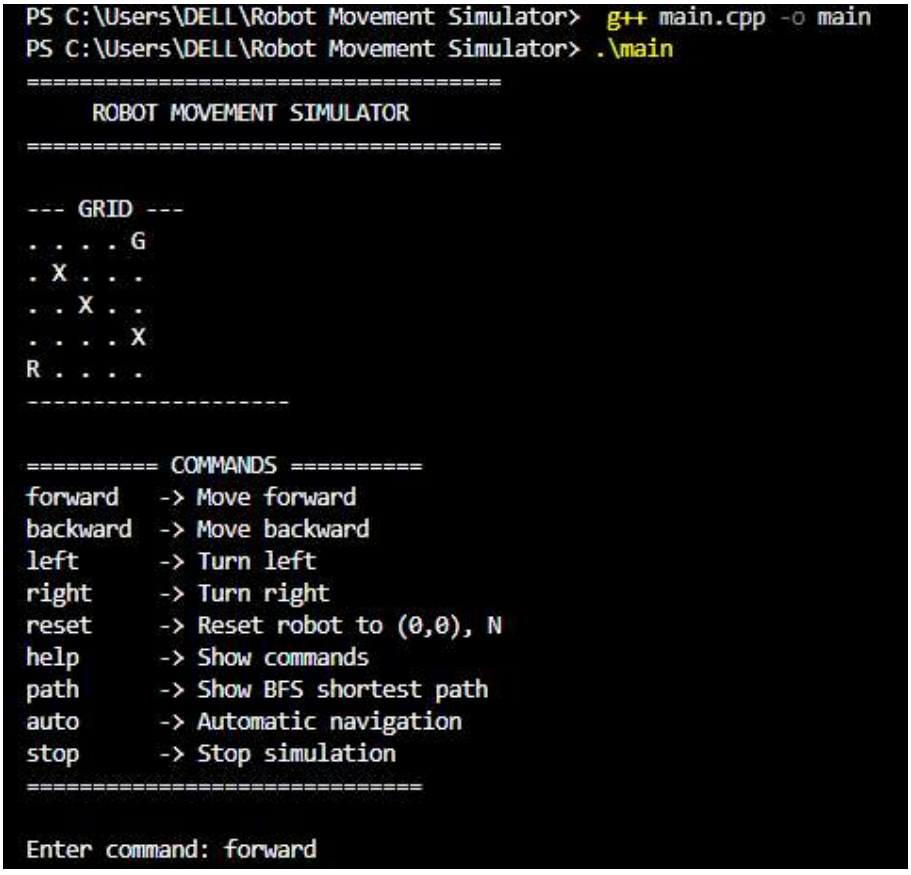
15. SCREENSHOTS

Screenshots are included to provide visual evidence of the implemented features and testing results of the Robot Movement Simulator. The screenshots show the simulator running in the console and demonstrate the robot's grid environment, navigation, sensor information, and final results.

15.1 Initial Grid and Commands

This screenshot shows the initial 5×5 grid, the starting robot position (0,0), the goal position (4,4), and the available commands displayed when the simulator starts.

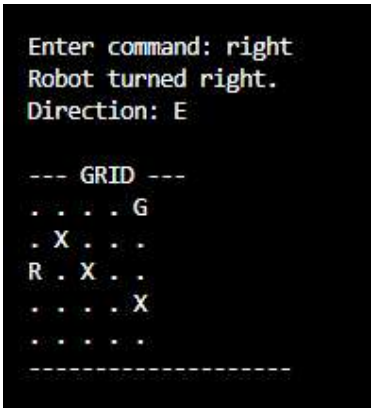
Figure 15.1: Initial Grid and Available Commands



15.2 Robot Movement and Direction

This screenshot shows the robot after performing movement and direction commands. It demonstrates the robot's current position and direction along with the updated grid.

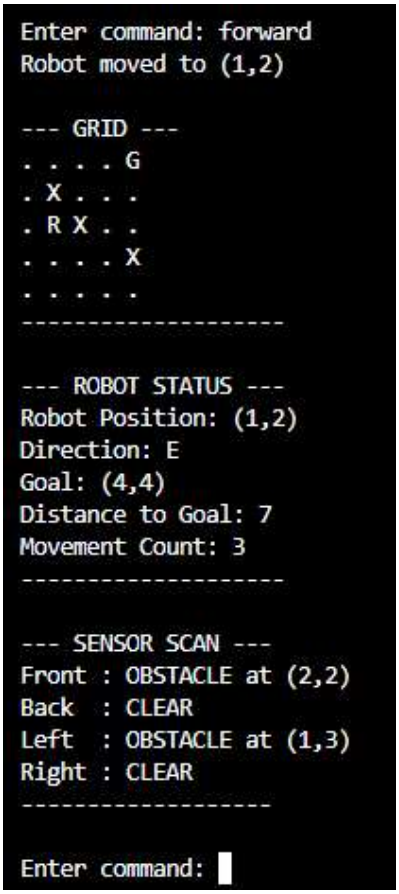
Figure 15.2: Robot Movement and Direction Control



15.3 Sensor Scan

This screenshot shows the simulated sensor information for Front, Back, Left, and Right based on the robot's current position and direction.

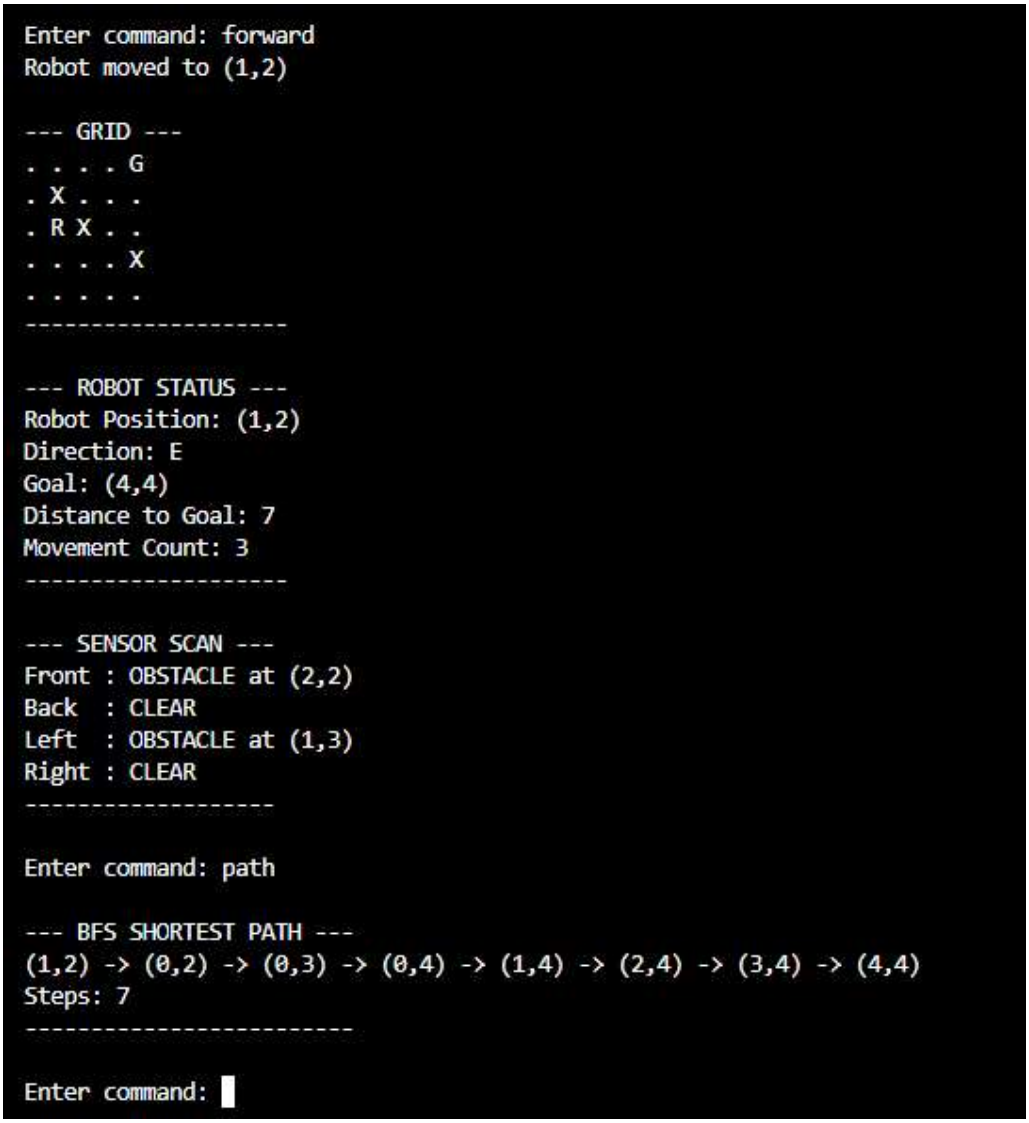
Figure 15.3: Sensor Scan Output



15.4 BFS Path Output

This screenshot shows the shortest valid path calculated using the BFS algorithm from the robot's current position to the goal.

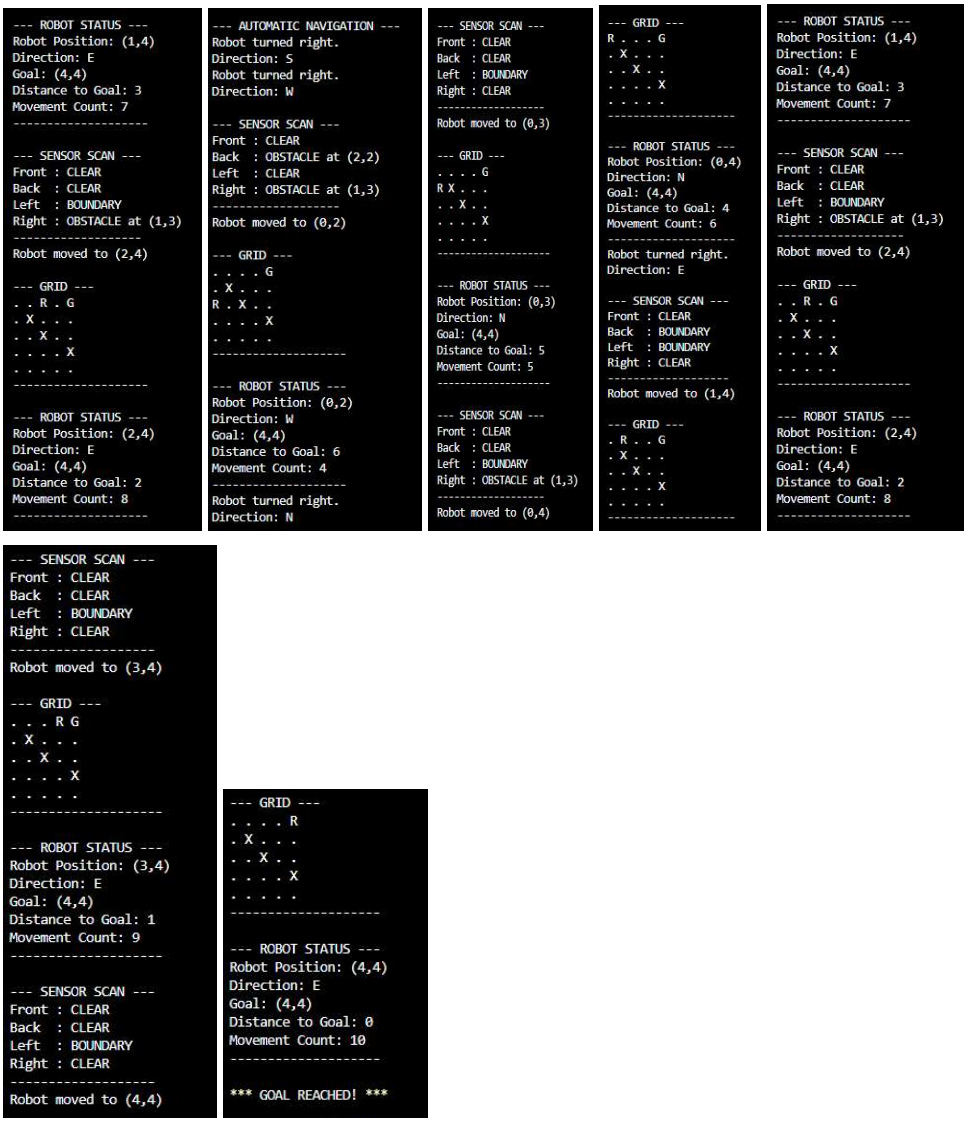
Figure 15.4: BFS Shortest Path Output



15.5 Automatic Navigation

This screenshot shows the robot performing automatic navigation using the auto command and following the BFS-based route toward the goal.

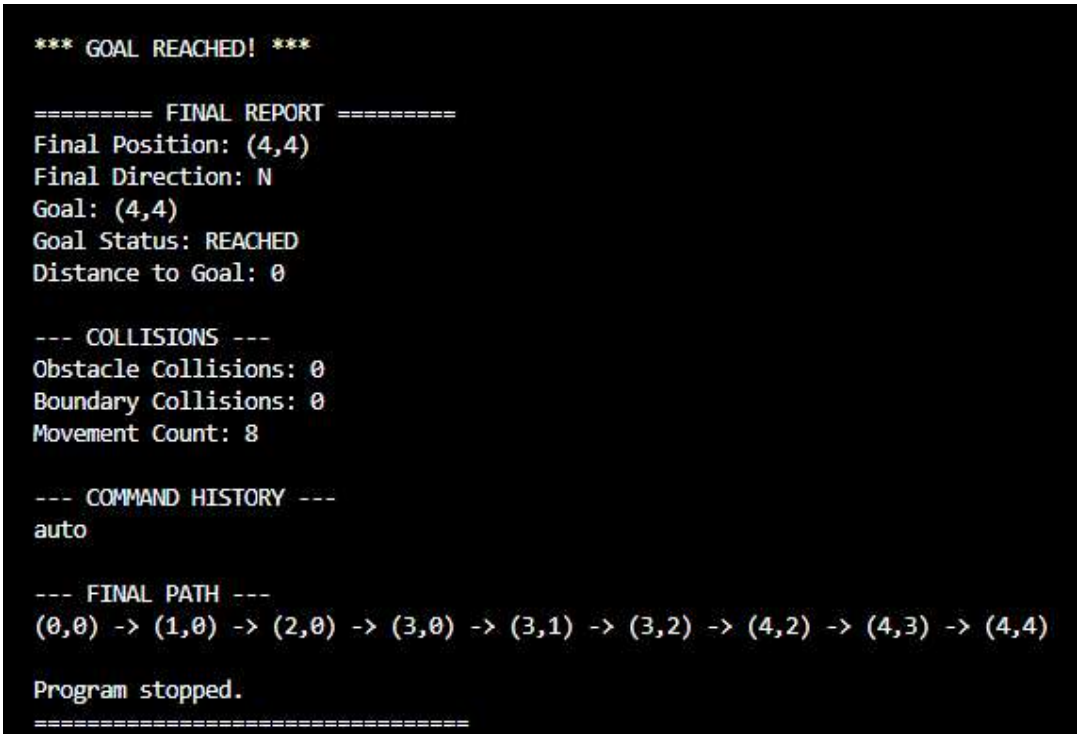
Figure 15.5: Automatic Navigation



15.6 Final Simulation Result

This screenshot shows the final navigation result after the robot reaches the goal. It includes the final position, direction, distance to the goal, movement count, collision statistics, command history, and final path.

Figure 15.6: Final Simulation Report



15.7 Screenshot Documentation

The screenshots presented in this section provide visual evidence of the Robot Movement Simulator's operation and testing results. They illustrate the initial grid, robot movement and direction, sensor scanning, BFS shortest-path output, automatic navigation, and final simulation report.

These screenshots help demonstrate the implemented features and verify the robot's navigation from the starting position (0,0) to the goal position (4,4).

16. CONCLUSION

The Robot Movement Simulator was developed as a C++-based project to understand and demonstrate fundamental concepts of robot navigation in a grid-based environment. The project successfully models a robot operating within a 5×5 grid containing predefined obstacles and boundaries.

The simulator implements essential robot navigation functions such as forward and backward movement, left and right direction control, obstacle detection, boundary

detection, simulated sensor scanning, BFS-based path planning, distance calculation, and automatic navigation.

The Breadth-First Search (BFS) algorithm was used to determine the shortest valid path to the predefined goal while considering obstacles and grid boundaries. The automatic navigation system uses this path to guide the robot step by step toward the goal.

The project also maintains movement count, obstacle collisions, boundary collisions, command history, and robot path information, and generates a final report when the simulation is stopped. Testing confirmed that the robot successfully navigated from (0,0) to (4,4) using 8 successful movements with zero obstacle collisions and zero boundary collisions.

Overall, this project provided practical experience in C++ programming, data structures, BFS path planning, sensor logic, robot movement, and grid-based navigation. It also provides a foundation for further learning and development in robotics and autonomous navigation.

16.1 Future Learning

The knowledge gained from this project can be used as a foundation for learning more advanced robotics concepts such as physical robot control, sensor integration, computer vision, and ROS-based robot navigation.

THANK YOU!